lab GCA session 3 Sample Report

thomasfeys10

February 2025

1 Part 1 Warps

The goal is to implement a kernel that inverts an image $x = 255 - x$, for each color component (RGB). Asses how the number of threads (full/partial warps) influences the computation time. Some example results are shown in the following figures. Fig. 1 shows the execution time including and excluding memory transfer in function of the number of threads per block. We see that the time goes down until a full warp is reached (32). This is to be expected because when using less then 32 threads per block, we are launching empty threads because they are scheduled per warp. Note that these results are obtained by running each kernel 50 times and averaging the execution times. A more complete view of all the runs can be seen in Fig. 2.

2 Striding

Note that the above code was run without striding i.e., the number of blocks was adjusted in order to get the correct number of threads. We can also implement striding to process all elements when we have fewer threads.

3 Part 2 Thread divergence

In this part we assess thread divergence by having a different execution path for the red values of the image. When processing the data in RGB, RGB, RGB, ... format each 'R'-thread in a warp will have to perform a longer (Gaussian

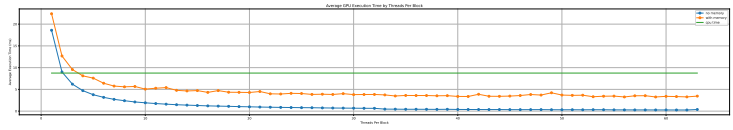


Figure 1: Avg execution times including and excluding memorytransfers

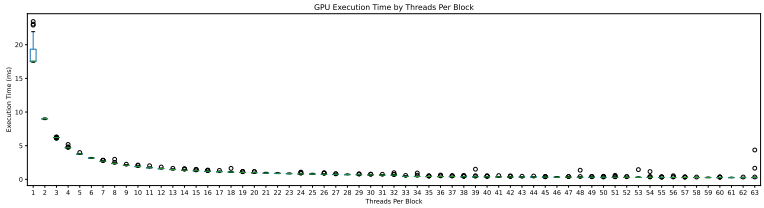


Figure 2: Boxplots of part 1 excluding memorytransfer

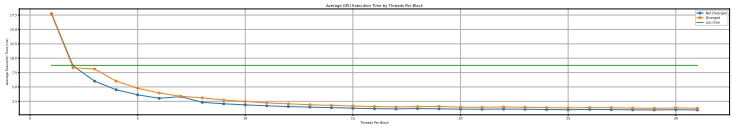


Figure 3: Divergent kernel Vs. non-divergent kernel

blurring) operation as compared to the image inversion that the 'G' and 'B'-threads need to do. This will leave the 'G' and 'B'-threads idle for a moment. For a fair comparison we can also process the image in a different format namely RRRR..., GGGG..., BBBB.... let's called this the clustered format. In this format we first process all R values, then all G values and so on. In this way all threads processing R values will have the same complexity hence, no divergence will occur leading to faster execution times as can be seen in Fig. 3.